

Patrón de Diseño State

Un patrón de comportamiento que permite a un objeto alterar su comportamiento cuando su estado interno cambia, como si el objeto cambiara su clase.



Recorrido por la Presentación

A continuación, exploraremos los conceptos clave del Patrón State, desde su definición hasta su aplicación práctica.

1

Introducción al Patrón State

2

El Problema: Máquinas de Estados Complejas

3

La Solución del Patrón State

4

Anatomía del Patrón State

5

Ejemplo Práctico: Reproductor de Medios

6

Aplicaciones y Mejores Prácticas

7

Ventajas, Desventajas y Relaciones

INTRODUCCIÓN

¿Qué es el Patrón State?

State es un **patrón de diseño de comportamiento** que permite a un objeto alterar su comportamiento cuando su estado interno cambia. Parece como si el objeto cambiara su clase.

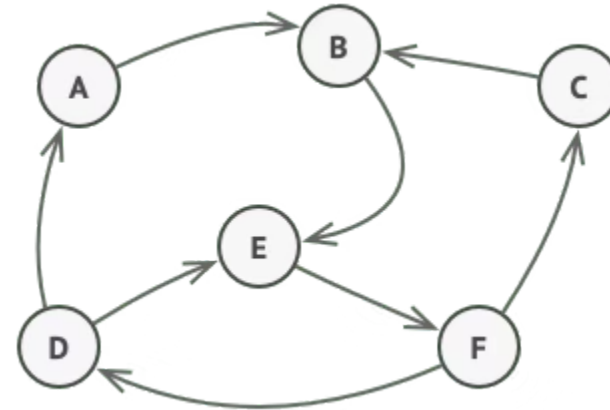
También conocido como **Estado**, este patrón encapsula los diferentes comportamientos asociados a cada estado en clases independientes, delegando el trabajo al objeto de estado activo en cada momento.

El Problema: Máquina de Estados Finitos

El patrón State está estrechamente relacionado con el concepto de la **Máquina de estados finitos**. La idea principal es que, en cualquier momento dado, un programa puede encontrarse en un **número finito de estados**.

Dentro de cada estado único, el programa se comporta de forma diferente y puede cambiar de un estado a otro instantáneamente. Sin embargo, dependiendo del estado actual, el programa puede cambiar o no a otros estados.

Estas normas de cambio, llamadas **transiciones**, también son finitas y predeterminadas.



Máquina de estados finitos.

PROBLEMA

Ejemplo: Clase Documento

Imagina que tienes una clase **Documento**. Un documento puede encontrarse en uno de estos tres estados: **Borrador**, **Moderación** y **Publicado**. El método `publicar` funciona de forma distinta en cada estado:

Borrador

Mueve el documento a moderación.

Moderación

Hace público el documento, pero sólo si el usuario actual es un administrador.

Publicado

No hace nada en absoluto.

PROBLEMA

Estados y Transiciones del Documento

El siguiente diagrama muestra los posibles estados y transiciones de un objeto de documento, ilustrando cómo el comportamiento cambia según el estado activo.

Posibles estados y transiciones de un objeto de documento.



Implementación con Condicionales

Las máquinas de estado se implementan normalmente con muchos operadores condicionales (if o switch) que seleccionan el comportamiento adecuado dependiendo del estado actual del objeto. ¿Te suena esta estructura?

```
class Document is
  field state: string
  // ...
  method publish() is
    switch (state)
      "draft":
        state = "moderation"
        break
      "moderation":
        if (currentUser.role == "admin")
          state = "published"
          break
      "published":
        // No hacer nada.
        break
    // ...
```

PROBLEMA

Debilidades del Enfoque Condicional

Condicionales monstruosos

La mayoría de los métodos contendrán condicionales enormes que eligen el comportamiento adecuado de acuerdo con el estado actual. Un código así es muy difícil de mantener.

Cambios costosos

Cualquier cambio en la lógica de transición puede requerir cambiar los condicionales de estado de **cada método**.

Crecimiento descontrolado

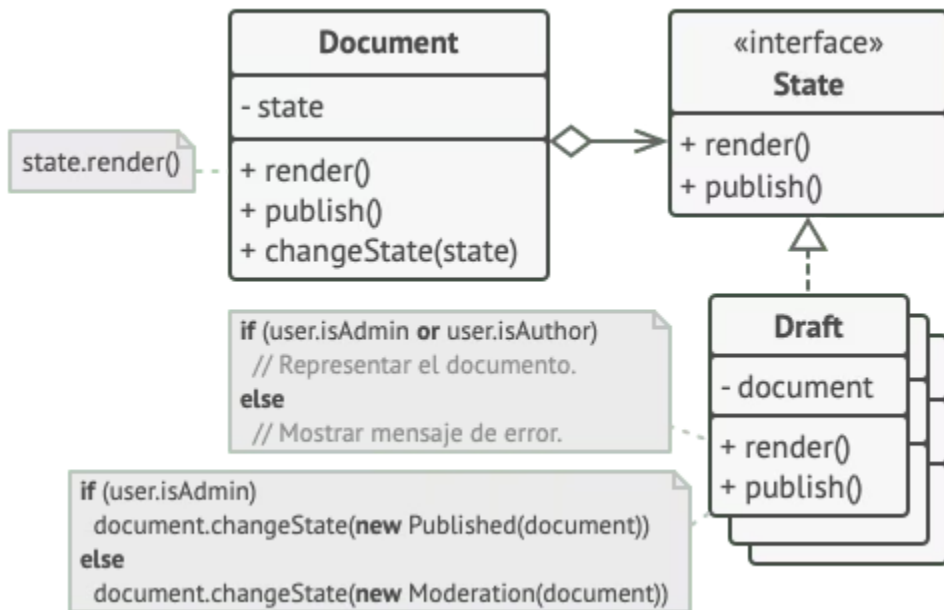
Es difícil predecir todos los estados y transiciones posibles en la etapa de diseño. Una máquina de estados esbelta puede crecer hasta convertirse en un abotargado desastre con el tiempo.

La Solución del Patrón State

El patrón State sugiere que crees **nuevas clases para todos los estados posibles** de un objeto y extraigas todos los comportamientos específicos del estado para colocarlos dentro de esas clases.

En lugar de implementar todos los comportamientos por su cuenta, el objeto original, llamado **contexto**, almacena una referencia a uno de los objetos de estado que representa su estado actual y **delega todo el trabajo** relacionado con el estado a ese objeto.

Delegación al Objeto de Estado



Para la transición del contexto a otro estado, se sustituye el objeto de estado activo por otro objeto que represente ese nuevo estado.

Esto sólo es posible si todas las clases de estado siguen la **misma interfaz** y el propio contexto funciona con esos objetos a través de esa interfaz.

Documento delega el trabajo a un objeto de estado.

SOLUCIÓN

State vs. Strategy

Esta estructura puede resultar similar al patrón **Strategy**, pero hay una diferencia clave:

Patrón State

Los estados particulares **pueden conocerse entre sí** e iniciar transiciones de un estado a otro.

Patrón Strategy

Las estrategias **casi nunca se conocen** entre sí. Son completamente independientes.

Analogía en el Mundo Real

Los botones e interruptores de tu **smartphone** se comportan de forma diferente dependiendo del estado actual del dispositivo:



Desbloqueado

Al pulsar botones se ejecutan varias funciones.



Bloqueado

Pulsar un botón desbloquea la pantalla.



Batería baja

Pulsar un botón muestra la pantalla de carga.

Componentes de la Estructura

1 Contexto

Almacena una referencia a uno de los objetos de estado concreto y le delega todo el trabajo específico del estado. Se comunica con el objeto de estado a través de la interfaz de estado y expone un setter para pasarle un nuevo objeto de estado.

2 Interfaz Estado

Declara los métodos específicos del estado. Estos métodos deben tener sentido para todos los estados concretos, porque no querrás que uno de tus estados tenga métodos inútiles que nunca son invocados.

3 Estados Concretos

Proporcionan sus propias implementaciones para los métodos específicos del estado. Pueden almacenar una referencia inversa al objeto de contexto para extraer información o iniciar transiciones de estado.

Transiciones de Estado

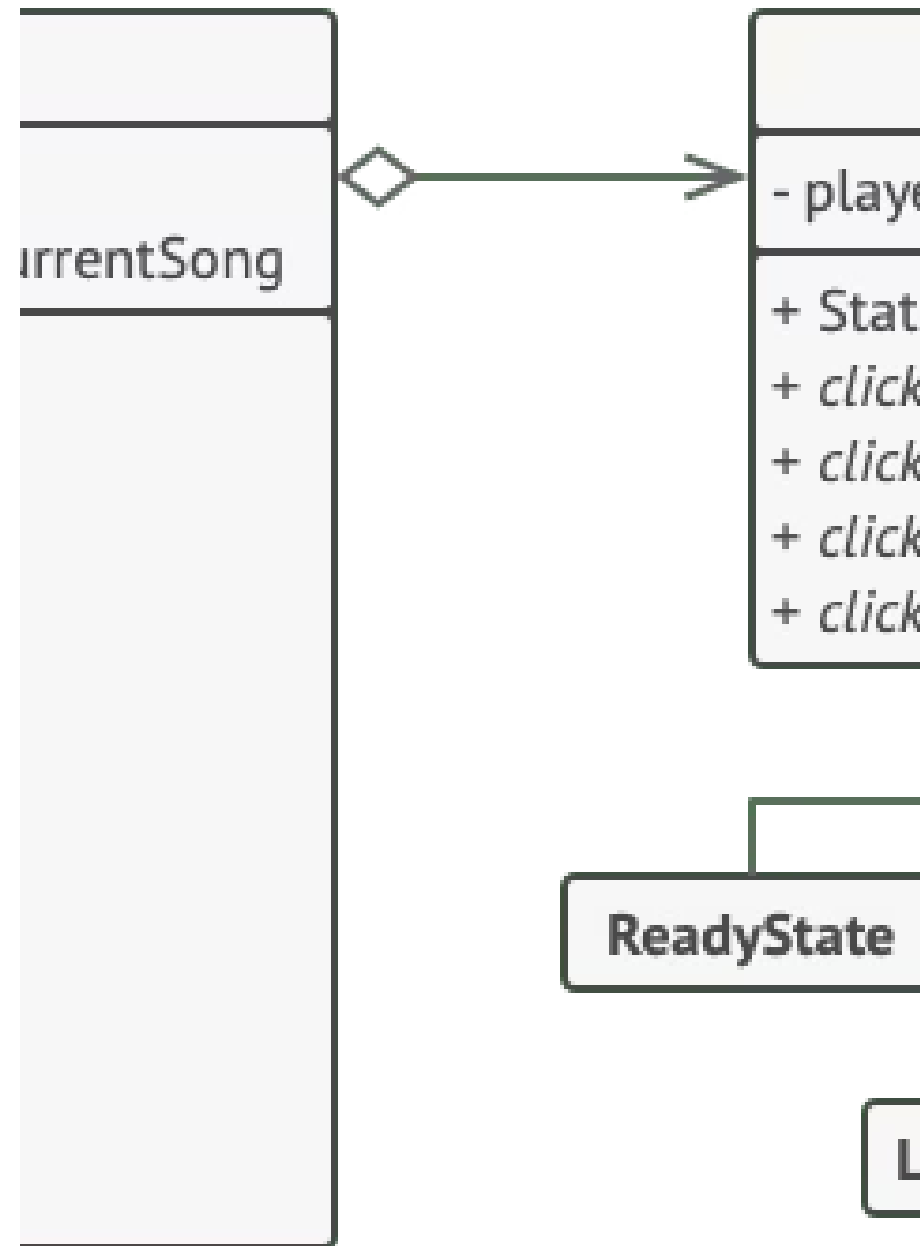
Los objetos de estado pueden almacenar una **referencia inversa** al objeto de contexto. A través de esta referencia, el estado puede extraer cualquier información requerida del objeto de contexto, así como iniciar transiciones de estado.

- ❏ Tanto el estado de contexto como el concreto pueden establecer el nuevo estado del contexto y realizar la transición de estado sustituyendo el objeto de estado vinculado al contexto.

Ejemplo: Reproductor de Medios

En este ejemplo, el patrón State permite a los mismos controles del reproductor de medios comportarse de forma diferente, dependiendo del estado actual de reproducción.

Ejemplo de cambio del comportamiento de un objeto con objetos de estado.



Clase `AudioPlayer` (Contexto)

El objeto principal del reproductor siempre está vinculado a un objeto de estado que realiza la mayor parte del trabajo. Algunas acciones sustituyen el objeto de estado actual, cambiando la forma en que el reproductor reacciona.

```
class AudioPlayer is
  field state: State
  field UI, volume, playlist, currentSong

  constructor AudioPlayer() is
    this.state = new ReadyState(this)

  UI = new UserInterface()
  UI.lockButton.onClick(this.clickLock)
  UI.playButton.onClick(this.clickPlay)
  UI.nextButton.onClick(this.clickNext)
  UI.prevButton.onClick(this.clickPrevious)

  method changeState(state: State) is
    this.state = state

  method clickLock() is
    state.clickLock()
  method clickPlay() is
    state.clickPlay()
  method clickNext() is
    state.clickNext()
  method clickPrevious() is
    state.clickPrevious()

  method startPlayback() is
    // ...
  method stopPlayback() is
    // ...
  method nextSong() is
    // ...
  method previousSong() is
    // ...
  method fastForward(time) is
    // ...
  method rewind(time) is
    // ...
```

PSEUDOCÓDIGO

Clase Base State

La clase estado base declara métodos que todos los estados concretos deben implementar, y proporciona una **referencia inversa** al objeto de contexto asociado. Los estados pueden utilizar esta referencia para dirigir el contexto a otro estado.

```
abstract class State is  
  protected field player: AudioPlayer
```

```
  constructor State(player) is  
    this.player = player
```

```
  abstract method clickLock()  
  abstract method clickPlay()  
  abstract method clickNext()  
  abstract method clickPrevious()
```

Estado Concreto: LockedState

Cuando desbloqueas un reproductor bloqueado, puede asumir uno de dos estados dependiendo de si estaba reproduciendo o no.

```
class LockedState extends State is
```

```
  method clickLock() is
```

```
    if (player.playing)
```

```
      player.changeState(new PlayingState(player))
```

```
    else
```

```
      player.changeState(new ReadyState(player))
```

```
  method clickPlay() is
```

```
    // Bloqueado, no hace nada.
```

```
  method clickNext() is
```

```
    // Bloqueado, no hace nada.
```

```
  method clickPrevious() is
```

```
    // Bloqueado, no hace nada.
```

Estado Concreto: ReadyState

El estado **ReadyState** representa al reproductor listo para iniciar la reproducción. Desde aquí se puede bloquear, reproducir o navegar entre canciones.

```
class ReadyState extends State is
```

```
method clickLock() is
```

```
player.changeState(new LockedState(player))
```

```
method clickPlay() is
```

```
player.startPlayback()
```

```
player.changeState(new PlayingState(player))
```

```
method clickNext() is
```

```
player.nextSong()
```

```
method clickPrevious() is
```

```
player.previousSong()
```

Estado Concreto: **PlayingState**

El estado **PlayingState** maneja la reproducción activa. Nótese cómo los botones de siguiente y anterior tienen comportamiento diferente según sea clic simple o doble clic.

```
class PlayingState extends State is
```

```
method clickLock() is
```

```
player.changeState(new LockedState(player))
```

```
method clickPlay() is
```

```
player.stopPlayback()
```

```
player.changeState(new ReadyState(player))
```

```
method clickNext() is
```

```
if (event.doubleclick)
```

```
player.nextSong()
```

```
else
```

```
player.fastForward(5)
```

```
method clickPrevious() is
```

```
if (event.doubleclick)
```

```
player.previous()
```

```
else
```

```
player.rewind(5)
```

¿Cuándo Usar el Patrón State?

1

Comportamiento dependiente del estado

Cuando tengas un objeto que se comporta de forma diferente dependiendo de su estado actual, el número de estados sea enorme y el código específico del estado cambie con frecuencia. Puedes añadir nuevos estados o cambiar los existentes independientemente, reduciendo el costo de mantenimiento.

2

Condicionales enormes

Cuando tengas una clase contaminada con enormes condicionales que alteran el modo en que se comporta la clase de acuerdo con los valores actuales de los campos. El patrón te permite extraer ramas de esos condicionales a métodos de las clases estado correspondientes.

3

Código duplicado

Cuando tengas mucho código duplicado por estados similares y transiciones de una máquina de estados basada en condiciones. El patrón te permite componer jerarquías de clases de estado y reducir la duplicación extrayendo el código común en clases abstractas base.

Cómo Implementarlo (Parte 1)

01

Definir el Contexto

Decide qué clase actuará como contexto. Puede ser una clase existente que ya tiene el código dependiente del estado, o una nueva clase si el código está distribuido a lo largo de varias clases.

02

Declarar la Interfaz de Estado

Aunque puede replicar todos los métodos declarados en el contexto, céntrate en los que pueden contener comportamientos específicos del estado.

03

Crear Clases de Estado

Para cada estado actual, crea una clase derivada de la interfaz de estado. Repasa los métodos del contexto y extrae todo el código relacionado con ese estado para meterlo en tu clase recién creada.

Cómo Implementarlo (Parte 2)

Al mover el código a la clase estado, puede que descubras que depende de miembros privados del contexto. Hay varias soluciones:

- Haz públicos esos campos o métodos.
- Convierte el comportamiento que estás extrayendo en un método público en el contexto e invócalo desde la clase de estado.
- Anida las clases de estado en la clase contexto (si tu lenguaje lo soporta).

1 Añadir referencia de estado

En la clase contexto, añade un campo de referencia del tipo de interfaz de estado y un setter público que permita sobrescribir el valor de ese campo.

2 Sustituir condicionales

Vuelve a repasar el método del contexto y sustituye los condicionales de estado vacíos por llamadas a métodos correspondientes del objeto de estado.

3 Cambiar el estado

Crea una instancia de una de las clases de estado y pásala a la clase contexto. Puedes hacer esto dentro del propio contexto, en distintos estados, o en el cliente.

Pros y Contras del Patrón State

✓ Ventajas

- **Principio de responsabilidad única.** Organiza el código relacionado con estados particulares en clases separadas.
- **Principio de abierto/cerrado.** Introduce nuevos estados sin cambiar clases de estado existentes o la clase contexto.
- **Simplifica el contexto** eliminando voluminosos condicionales de máquina de estados.

✗ Desventajas

Aplicar el patrón puede resultar **excesivo** si una máquina de estados sólo tiene unos pocos estados o raramente cambia.

Relaciones con Otros Patrones

Bridge, State, Strategy y Adapter

Tienen estructuras muy similares. Todos se basan en la **composición**, que consiste en delegar trabajo a otros objetos. Sin embargo, todos solucionan problemas diferentes. Un patrón no es simplemente una receta para estructurar tu código, también puede comunicar a otros desarrolladores el problema que resuelve.

State como extensión de Strategy

Ambos patrones se basan en la composición: cambian el comportamiento del contexto delegando parte del trabajo a objetos ayudantes. **Strategy** hace que estos objetos sean completamente independientes y no se conozcan entre sí. Sin embargo, **State** no restringe las dependencias entre estados concretos, permitiéndoles alterar el estado del contexto a voluntad.